093625

Design and Analysis of Algorithms

CO1- Understanding of complexity representation in terms of Big O, theta and Omega notations.
CO2- Derive and solve recurrences describing the performance of Sorting and Divide-and-Conquer algorithm
CO3- Compare and analyze different advance data structures such as RB tree, B Tree, Binomial Heap, Fibonacci Heap
CO4- Explain the major graph algorithm and their analyses. Employ graphs to model various problems, when appropriate and their analysis
CO5- Understand the greedy paradigm and able to analyze when an algorithmic design situation calls for it. Synthesize greedy algorithm
CO6- Synthesize dynamic programming algorithms and analyze them
CO7- Understand the backtracking and branch and bound paradigm and able to analysis when an algorithmic design situation calls for it
CO8- Understand the complexity theory with P, NP, NP-complete, NP-Hard

Printed Pages: 4 **University Roll No. .....**

Mid Term Examination, Odd Semester 2025-26
B.Tech. (CS+AIML+BCT+CCV+CSF+DA), 3 Year, V Semester
BCSC 1012 & Design & Analysis of Algorithms

Time: 2 Hours **Maximum Marks: 30**

Instruction for students:
1. Attempt all questions

Section – A

*Attempt **All** Questions* 3 X 5 = 15 Marks

No.	Detail of Question	Marks	CO	BL	KL
1	Consider a situation where swap operation is very costly. Which of the following sorting algorithm should be preferred so that the number of swap operations are minimized in general? Write the algorithm of quick sort. (i) Quick Sort (ii) Selection Sort (iii) Insertion Sort (iv) Merge Sort	3	2	An	C
2	During a code review, your colleague writes a recursive function with the recurrence relation $T(n) = T(n-2) + \log n$, $T(0) = 0$. They argue that the algorithm has linear time complexity. Task:	3	2	An	P

	(a) Use the iterative method to expand and find the actual time complexity. (b) Explain how such incorrect assumptions could impact system scalability and suggest a better approach.				

3	A company is developing a data processing module that involves multi-level iteration to calculate certain statistics. The following function fun1() is part of the performance-critical section of the application. Analyze the total time and time complexity of fun1().
	
	int fun1 (int n)
	{
	int i, j, k, p, q = 0;
	for (i = 1; i < n; ++i)
	{
	p = 0;
	for (j = n; j > 1; j = j/2)
	++p;
	for (k = 1; k < p; k = k*2)
	++q;
	}
	return q;
	}
		3	1	An	C

4	A company is analyzing the execution time of two different algorithms: **Algorithm A** has a running time modeled by $g_1(n)$ **Algorithm B** has a running time modeled by $g_2(n)$ $g_1(n) = \begin{cases} n^3 \text{ for } 0 \le n < 10,000 \\ n^2 \text{ for } n \ge 10,000 \end{cases}$ $g_2(n) = \begin{cases} n \text{ for } 0 \le n \le 100 \\ n^3 \text{ for } n > 100 \end{cases}$ The company wants to determine which algorithm is asymptotically more efficient and in what ranges one algorithm should be preferred over the other. Justify your answer with proper explanation.	3	1	E	C

5	A database engineer is working on optimizing search and storage operations for a large system. The engineer decides to use a B-Tree (degree = 2) because of its efficiency in handling disk-based access, ensuring balanced search, insertion, and deletion operations. (a) The engineer is asked to identify three real-world application areas where the B-Tree data structure is applied. (b) To test the concept, the engineer performs insertion operations on a B-Tree with degree 2 for the following sequence of keys:7, 4, 13, 8, 12, 3, 6, 9, 11	3	3	U	C

Section – B

*Attempt **All** Questions* 5 X 3 = 15 Marks

No.	Detail of Question	Marks	CO	BL	KL
6	A university timetable scheduling system stores classes in a BST based on starting times. If classes are inserted in ascending order of time, explain the problem that arises and how an AVL Tree solves it, Explain the key properties and characteristics of Red-Black Trees. Perform an insertion operation using a suitable algorithm in Red-Black Tree on the following string "ALGORITHMS".	5	3	A	P
7	An e-commerce website uses Heap Sort to prioritize orders based on delivery urgency. They want to optimize for minimum heapify calls. (a) Explain how bottom-up heap construction can reduce the number of heapify operations compared to inserting elements one-by-one. (b) If the array has 1,000,000 elements, estimate the number of heapify calls for both methods. (c) Which method is more efficient in terms of time complexity, and why? (d) Write down the algorithm of Heap sort.	5	2	E	C

8	Imagine you are developing a task scheduler for a multi-threaded application. Each task has a priority associated with it, and tasks with higher priorities should be executed before tasks with lower priorities. Explain how you can use a Binomial Heap to implement the priority queue for scheduling tasks efficiently. Construct a Binomial Heap for the given list of elements: 7, 2, 4, 17, 1, 11, 6, 8, 15, 10, 20.	5	3	U	C